

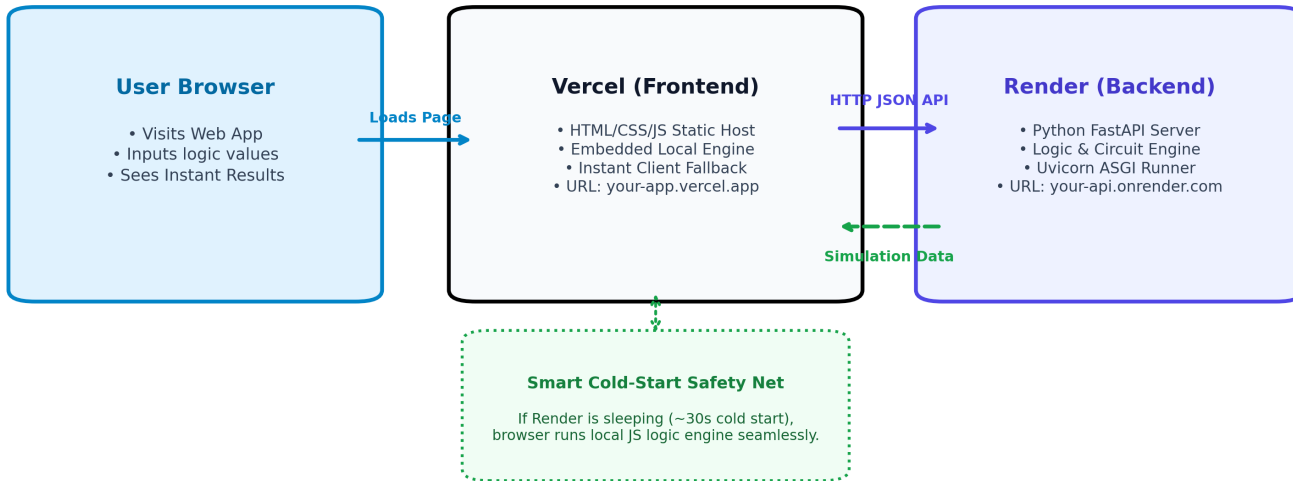
Full Deployment & Architecture Plan

Digital Logic Simulation Suite

Architecture, Step-by-Step Implementation, Visual Charts & Operations Guide

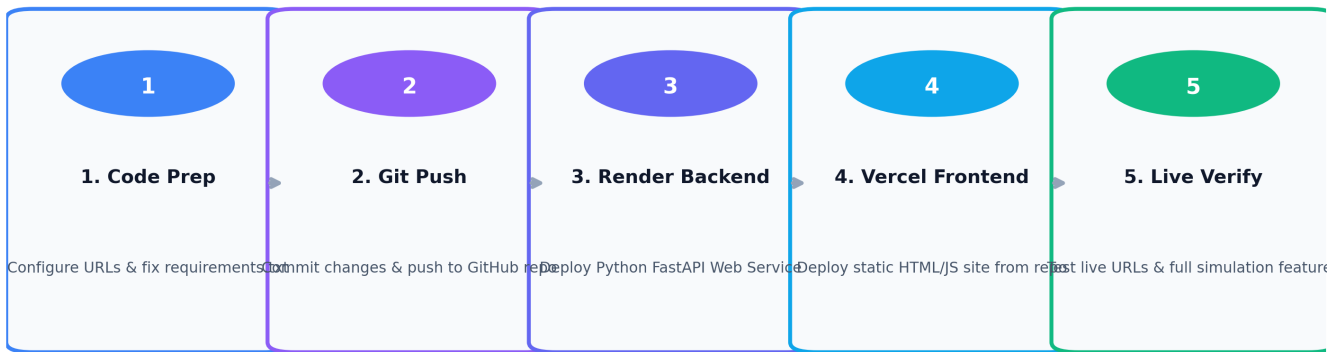
1. Architecture & Live Flow Overview

Your Digital Logic Simulation Suite is structured into a decoupled frontend and backend. The diagram below illustrates how users interact with the app, how API requests route between Vercel and Render, and how the built-in Client-Side Fallback Engine guarantees 100% uptime even if Render is waking up from sleep.



2. End-to-End Deployment Roadmap

Deploying your application is split into 5 clear chronological stages. Follow each step sequentially:



Zero Breaking Changes Guarantee

The deployment plan only replaces hardcoded localhost strings with a unified config variable and fixes requirements.txt. All logic circuits, simulation math, and UI layouts remain 100% intact.

3. Detailed Code Preparation Matrix (8 Changes)

Review each required file modification below with exact target locations, what is being updated, and the technical rationale.

3.1. Central API Endpoint Configuration

NEW FILE

Target: frontend/config.js

What: Creates a global variable 'const API_BASE = "https://YOUR-APP.onrender.com";'.

Why: Consolidates the backend URL into one single file. When deployed, you only update this one string instead of editing dozens of individual files.

3.2. Remove Hardcoded Port in Circuit Viewer

EDIT

Target: frontend/circuit-viewer.js (Line 11)

What: Removes 'const API_BASE = "http://127.0.0.1:8000";' in the viewer engine.

Why: Eliminates variable shadowing so the circuit viewer engine automatically uses the centralized live API_BASE from config.js.

3.3. Update Code Converter Host Resolution

EDIT

Target: frontend/converter-engine.js (4 places)

What: Replaces localhost-seeking arrays in all 4 async functions with 'const hosts = [API_BASE].filter(Boolean);'.

Why: Directs all code conversion requests (Gray, BCD, Excess-3) to your live Render server while preserving the instant client-side fallback.

3.4. Update Standalone App Module Fetch

EDIT

Target: frontend/app.js (Line 11)

What: Changes 'http://127.0.0.1:8000/convert' to `\${API\_BASE}/convert`.

Why: Ensures the auxiliary conversion forms route correctly to the production cloud endpoint.

3.5. Include Config Script Across All HTML Interfaces

EDIT

Target: All HTML files in frontend/ (11 files)

What: Adds '<script src="config.js"></script>' in the <head> before other script tags across index.html, comparator.html, multiplexer.html, adder-subtractor.html, schematic.html, etc.

Why: Guarantees that the API_BASE URL is instantiated in the browser's global scope before any simulator scripts execute.

3.6. Streamline Python Dependencies

EDIT

Target: requirements.txt

What: Replaces unused data-science packages (pandas, scikit-learn, seaborn, streamlit) with only 'fastapi' and 'uvicorn[standard]'.

Why: Reduces Render container build time from ~3 minutes down to under 20 seconds and prevents dependency conflicts during cloud container initialization.

3.7. Python Package Anchor

NEW FILE

Target: backend/__init__.py

What: Creates a standard Python package initialization file.

Why: Ensures Python's module resolver correctly loads internal circuit routers (circuit_router, mux_router, convert, compare, arithmetic) on cloud Linux servers.

3.8. Git Ignore Rules

NEW FILE

Target: .gitignore

What: Configures ignore rules for __pycache__/, *.pyc, .pytest_cache/, server.log, and .vscode/.

Why: Keeps the GitHub repository clean and prevents local bytecode/logs from creating deployment noise.

4. Step-by-Step Render Deployment (Backend API)

Follow these exact steps to host your Python FastAPI service on Render:

- **Stage 1: Push Code:**
Commit all prepped code changes to GitHub using: `git add -A && git commit -m 'Deploy config' && git push origin main`
- **Stage 2: Render Account:**
Go to <https://render.com> and sign up / log in with your GitHub account.
- **Stage 3: Create Service:**
From Dashboard, click 'New +' -> 'Web Service' and select your 'logic_sim_suite' repository.
- **Stage 4: Enter Settings:**
Fill out the exact build and runtime configuration provided in the table below:

Render Field Name	Recommended Value / Setting
Name	logic-sim-backend
Region	Singapore (or closest to your location)
Branch	main
Runtime	Python
Build Command	pip install -r requirements.txt
Start Command	cd backend && uvicorn main:app --host 0.0.0.0 --port \$PORT
Instance Type	Free (\$0.00 / month)

How to Verify Backend Health

Once deployed, copy your Render URL (e.g. <https://logic-sim-backend.onrender.com>) and append '/docs' in your browser. You should see the interactive FastAPI Swagger UI displaying all endpoints (/convert, /mux, /compare, /add-subtract, /circuit/).

5. Step-by-Step Vercel Deployment (Frontend UI)

Follow these steps to host your static website and circuit visualizer on Vercel CDN:

- **Stage 1: Set Live URL:**
In frontend/config.js, replace 'YOUR-APP-NAME' with your actual Render URL and push to GitHub.
- **Stage 2: Vercel Account:**
Visit <https://vercel.com> and log in with GitHub.
- **Stage 3: Import Project:**
Click 'Add New...' -> 'Project', choose 'logic_sim_suite' from your repo list.
- **Stage 4: Set Root Directory:**
Crucial: Click 'Edit' next to Root Directory and select the 'frontend' folder.
- **Stage 5: Deploy:**
Click 'Deploy'. Your frontend will build and be live worldwide in ~20-30 seconds!

Vercel Field Name	Recommended Value / Setting
Framework Preset	Other
Root Directory	frontend (click Edit to select)
Build Command	(leave empty - static files)
Output Directory	. (current directory)

6. Operations & Troubleshooting Decision Tree

If you encounter any behavior issues after deployment, refer to this quick diagnostic matrix:

Issue: CORS Blocked

FastAPI middleware allows origins ['*']
Check if API_BASE URL has https://

Issue: 404 On Vercel

Ensure Vercel Root Directory
is set to 'frontend'

Issue: Backend Sleeping

Free Render spins down after 15m.
Client JS fallback keeps app working!

7. Pre-Flight & Post-Launch Verification Checklist

Verification Scope	Quality Assurance Check Item
[] Pre-Flight	requirements.txt contains only 'fastapi' and 'uvicorn[standard]'
[] Pre-Flight	frontend/config.js is loaded in all 11 HTML files
[] Pre-Flight	backend/__init__.py and .gitignore exist in project root
[] Backend Test	Render /docs Swagger UI loads without errors
[] Backend Test	/convert POST request returns status 200 with valid binary JSON
[] Frontend Test	Vercel live URL renders home page, cards, and navigation links
[] Integration Test	KiCad Schematic Viewer interactive bit-toggles update wire paths live
[] Integration Test	Code converters (Binary, Gray, BCD, Excess-3) output matching truth tables
[] Fall-back Test	Frontend operates gracefully even if backend takes 30s to cold-start

Maintenance & Future Updates

Whenever you make future updates to your simulation logic or UI, simply push your commit to GitHub ('git push origin main'). Both Render and Vercel will automatically detect the changes and redeploy your website with zero downtime!